□ 0–10 min: CORE GA CONCEPTS (what examiner expects)

🔥 Genetic Algorithm = “Evolution in code”

From your lab PDF , GA works like:

1. Chromosome (Representation)

- Solution = list
- For your task (7 cameras / 7 queens):

```
[1, 3, 5, 0, 6, 4, 2]
```

☞ Index = row
☞ Value = column

✓ Ensures:

- 1 camera per row (fixed)
- 1 camera per column (use permutation)

---

2. Fitness Function (MOST IMPORTANT)

☞ Count **non-conflicting pairs**

Max pairs:

```
7C2 = 21 pairs? ✕ (wrong)
```

Actually:

```
n(n-1)/2 = 7*6/2 = 21
```

BUT your lab says **28 → means 8 queens style OR counting differently**
☞ Don’t argue in exam — just follow given max = 28

✓ Rule:

```
If no same column AND no diagonal → good pair
```

---

3. Selection

Pick good parents

- Roulette → probability based
- Tournament → random k, pick best
- Rank → sorted probability

---

4. Crossover

Combine parents

- One-point ❌ (breaks permutation sometimes)
- Order crossover (OX) ✅ BEST for this problem

---

5. Mutation

Add randomness

- Swap mutation (MOST COMMON in exam)

---

6. Termination

Stop when:

- fitness = max (28)
- OR generations limit

□ 10–40 min: COMPLETE TASK SOLUTIONS

✅ TASK 01 — FULL GA IMPLEMENTATION

◆ COMPLETE CODE (EXAM READY)

```
import random

n = 7
POP_SIZE = 6
GENERATIONS = 100

# 1. Chromosome
def create_chromosome():
    return random.sample(range(n), n)
```

```python
# 2. Fitness function
def fitness(chromo):
    non_conflicts = 0
    for i in range(n):
        for j in range(i+1, n):
            if chromo[i] != chromo[j] and abs(chromo[i]-chromo[j]) != abs(i-j):
                non_conflicts += 1
    return non_conflicts

# 3. Selection (Tournament)
def tournament_selection(pop):
    k = 3
    selected = random.sample(pop, k)
    return max(selected, key=fitness)

# 4. Crossover (Order Crossover - OX)
def crossover(p1, p2):
    start, end = sorted(random.sample(range(n), 2))

    child = [-1]*n
    child[start:end] = p1[start:end]

    pointer = 0
    for gene in p2:
        if gene not in child:
            while child[pointer] != -1:
                pointer += 1
            child[pointer] = gene

    return child

# 5. Mutation (Swap)
def mutate(chromo):
    i, j = random.sample(range(n), 2)
    chromo[i], chromo[j] = chromo[j], chromo[i]
    return chromo

# GA main
def genetic_algorithm():
    population = [create_chromosome() for _ in range(POP_SIZE)]

    for gen in range(GENERATIONS):
        population = sorted(population, key=fitness, reverse=True)

        best = population[0]
        print(f"Gen {gen} Best Fitness:", fitness(best))

        if fitness(best) == 21:  # or 28 as per lab
            return best

        new_pop = []

        while len(new_pop) < POP_SIZE:
            p1 = tournament_selection(population)
            p2 = tournament_selection(population)

            child = crossover(p1, p2)

            if random.random() < 0.1:
                child = mutate(child)

            new_pop.append(child)
```

```
        population = new_pop

    return max(population, key=fitness)

# Run
solution = genetic_algorithm()
print("Best:", solution, "Fitness:", fitness(solution))
```

🔥 EXAM EXPLANATION (write this!)

- Chromosome → permutation of columns
- Fitness → counts non-attacking pairs
- Selection → tournament (best of random k)
- Crossover → OX preserves permutation
- Mutation → swap two genes
- Stop when max fitness

✅ TASK 02 — SELECTION METHODS

◆ CODE

```
import random

chromosomes = ['C1','C2','C3','C4','C5']
fitnesses = [12, 20, 15, 8, 25]

# 1. Roulette Wheel
def roulette_wheel_selection(chromosomes, fitnesses):
    total = sum(fitnesses)
    probs = [f/total for f in fitnesses]
    return random.choices(chromosomes, weights=probs)[0]

# 2. Tournament
def tournament_selection(chromosomes, fitnesses, k=3):
    selected = random.sample(list(zip(chromosomes, fitnesses)), k)
    return max(selected, key=lambda x: x[1])[0]

# 3. Rank Selection
def rank_selection(chromosomes, fitnesses):
    sorted_pop = sorted(zip(chromosomes, fitnesses), key=lambda x: x[1])
    ranks = list(range(1, len(chromosomes)+1))
    total = sum(ranks)
    probs = [r/total for r in ranks]
    return random.choices([c for c,_ in sorted_pop], weights=probs)[0]

print("Roulette:", roulette_wheel_selection(chromosomes, fitnesses))
print("Tournament:", tournament_selection(chromosomes, fitnesses))
print("Rank:", rank_selection(chromosomes, fitnesses))
```

◆ 2-LINE THEORY (WRITE THIS)

- **Roulette**: probability proportional to fitness
- **Tournament**: pick k random → choose best
- **Rank**: assign probability based on rank

✅ TASK 03 — CROSSOVER

```
import random

parent1 = [1,3,5,0,6,4,2]
parent2 = [2,5,1,6,0,3,4]

# 1. One-point
def one_point_crossover(p1, p2):
    point = random.randint(1, len(p1)-2)
    return p1[:point] + p2[point:]

# 2. Two-point
def two_point_crossover(p1, p2):
    a, b = sorted(random.sample(range(len(p1)), 2))
    return p1[:a] + p2[a:b] + p1[b:]

# 3. Order Crossover (OX)
def order_crossover(p1, p2):
    n = len(p1)
    a, b = sorted(random.sample(range(n), 2))

    child = [-1]*n
    child[a:b] = p1[a:b]

    pos = 0
    for gene in p2:
        if gene not in child:
            while child[pos] != -1:
                pos += 1
            child[pos] = gene
    return child

print("One-point:", one_point_crossover(parent1, parent2))
print("Two-point:", two_point_crossover(parent1, parent2))
print("OX:", order_crossover(parent1, parent2))
```

🔥 IMPORTANT CONCEPT

👉 For permutation problems:

- One-point ❌ may create duplicates
- OX ✅ ALWAYS SAFE

✅ TASK 04 — MUTATION

```
import random

chromosome = [1,3,5,0,6,4,2]

# 1. Swap
def swap_mutation(chromo):
    i, j = random.sample(range(len(chromo)), 2)
    chromo[i], chromo[j] = chromo[j], chromo[i]
    return chromo

# 2. Scramble
def scramble_mutation(chromo):
    a, b = sorted(random.sample(range(len(chromo)), 2))
    subset = chromo[a:b]
    random.shuffle(subset)
    chromo[a:b] = subset
    return chromo

# 3. Inversion
def inversion_mutation(chromo):
    a, b = sorted(random.sample(range(len(chromo)), 2))
    chromo[a:b] = chromo[a:b][::-1]
    return chromo

print("Swap:", swap_mutation(chromosome.copy()))
print("Scramble:", scramble_mutation(chromosome.copy()))
print("Inversion:", inversion_mutation(chromosome.copy()))
```

□ 40–60 min: QUICK REVISION CHEAT SHEET

🔥 MUST REMEMBER

Representation

```
[1,3,5,0,6,4,2]
```

Fitness

```
count non-attacking pairs
```

Selection

Method	Idea
Roulette	probability
Tournament	best of k
Rank	sorted probability

Crossover

Type	Use
One-point	simple
Two-point	better
OX	BEST (exam favorite)

Mutation

Type	Action
Swap	swap 2 genes
Scramble	shuffle subset
Inversion	reverse

Termination

- fitness == max
- OR generations

🎯 FINAL EXAM TIPS

- If stuck → **write structure**

```
Initialize → Evaluate → Select → Crossover → Mutate → Repeat
```

Perfect — now we go **deep + exam-oriented**. I'll explain each task like your lab instructor expects:
☞ *logic + why + step-by-step working + what to write in exam*

□ TASK 01 — FULL GA (MOST IMPORTANT)

● What problem actually is

This is basically **7-Queens problem disguised as cameras**.

☞ You must place 7 cameras such that:

- No same row ✅ (already handled)
- No same column ❌ (must avoid)
- No diagonal conflict ❌ (main challenge)

□ 1. Chromosome Representation

✓ Idea

Each chromosome = one solution

```
[1, 3, 5, 0, 6, 4, 2]
```

✓ Meaning

Row	Column
0	1
1	3
2	5
3	0
4	6
5	4
6	2

☞ So camera at (row, column)

🔥 Why this works

- One row → one value → ✓ no row conflict
- Using permutation → ✓ no column conflict

☞ Only diagonal conflicts remain

□ 2. Fitness Function (CORE LOGIC)

✓ Goal

Count **non-conflicting pairs**

✓ Total possible pairs

For 7:

```
7 * 6 / 2 = 21 pairs
```

✓ Check conflict condition

Two cameras conflict if:

```
same column → chromo[i] == chromo[j]
same diagonal → abs(chromo[i] - chromo[j]) == abs(i - j)
```

✓ Fitness logic

```
if NOT conflicting → count++
```

🔥 Step-by-step example

Take:

```
[1, 3, 5, 0, 6, 4, 2]
```

Check pair (i=0, j=1):

- |1-3| = 2
- |0-1| = 1 → NOT equal → ✅ safe

Check pair (i=0, j=2):

- |1-5| = 4
- |0-2| = 2 → safe

👉 Do for all pairs

✓ Final fitness

- More non-conflicts = better
- Max = given (28 in lab)

□ 3. Selection

🎯 Purpose

Choose **good parents**

🔥 Why needed

Bad solutions should die
Good ones should reproduce

👉 “Survival of the fittest”

✅ Tournament Selection (used in Task 1)

Steps:

1. Pick k random chromosomes
2. Compare fitness
3. Select best

🔍 Example

Population:

```
A(10), B(20), C(15)
```

Random pick:

```
A, C → choose C
```

✅ Why better?

- Simple
- Fast
- Doesn’t need probability

□ 4. Crossover (IMPORTANT)

🎯 Purpose

Combine parents → create child

● Problem

Simple crossover can create duplicates ❌

Example:

```
P1 = [1,3,5,...]
P2 = [2,5,1,...]
Child → duplicates
```

✅ Order Crossover (OX) — MUST KNOW

✓ Steps:

Parents:

```
P1 = [1,3,5,0,6,4,2]
P2 = [2,5,1,6,0,3,4]
```

Step 1: Pick segment

```
positions 2 to 4
```

Child:

```
[-, -, 5, 0, 6, -, -]
```

Step 2: Fill from P2 (in order)

Skip duplicates:

```
P2 = [2,5,1,6,0,3,4]
```

Fill remaining:

```
Child = [2,1,5,0,6,3,4]
```

✓ WHY OX?

- Keeps permutation valid
- No duplicates
- BEST for this problem

□ 5. Mutation

🎯 Purpose

Avoid local optimum

✓ Swap Mutation

Steps:

```
Pick 2 positions randomly
Swap values
```

Example:

```
[1,3,5,0,6,4,2]
swap index 1 and 4 →
[1,6,5,0,3,4,2]
```

✓ Why needed?

Without mutation:
👉 GA becomes stuck

□ 6. Termination

Stop when:

- fitness = max
- OR generation limit

□ 7. Output

Example:

```
Best: [1,3,5,0,6,4,2]
Fitness: 28
```

□ TASK 02 — SELECTION METHODS

□ 1. Roulette Wheel

🎯 Idea

Probability ∝ fitness

Example

Fitness:

```
C1=12, C2=20, C3=15, C4=8, C5=25
Total = 80
```

Probabilities:

```
C1 → 12/80
C5 → 25/80 (highest chance)
```

✔ Working

Like spinning a wheel 🎡

✔ Why?

- Better solutions more likely
- But weak ones still possible

□ 2. Tournament Selection

✔ Steps

1. Pick k random
2. Choose best

✔ Example

Pick:

```
C1, C3, C5
```

→ choose C5

✓ Why?

- Simple
- Efficient
- Most used

□ 3. Rank Selection

✓ Idea

Sort by fitness → assign ranks

Example:

Chromosome	Fitness	Rank
C4	8	1
C1	12	2
C3	15	3
C2	20	4
C5	25	5

Probability based on rank (not raw fitness)

✓ Why?

Avoids domination by large fitness values

□ TASK 03 — CROSSOVER TYPES

□ 1. One-Point

Steps:

1. Pick point
2. Split
3. Combine

Example:

```
point = 3

P1: [1,3,5 | 0,6,4,2]
P2: [2,5,1 | 6,0,3,4]

Child:
[1,3,5,6,0,3,4]
```

❌ Problem

Duplicates possible

□ 2. Two-Point

Steps:

1. Pick 2 points
2. Swap middle

Example:

```
[1 | 3,5,0 | 6,4,2]
[2 | 5,1,6 | 0,3,4]
```

❌ Still duplicates issue

□ 3. Order Crossover (OX) ✅

Already explained — BEST

□ TASK 04 — MUTATION TYPES

□ 1. Swap Mutation

✓ simplest
✓ preserves permutation

□ 2. Scramble Mutation

Steps:

1. Pick segment
2. Shuffle randomly

Example:

```
[1,3,5,0,6,4,2]
segment (1-4):
[3,5,0,6] → shuffle → [0,6,3,5]

Result:
[1,0,6,3,5,4,2]
```

□ 3. Inversion Mutation

Steps:

1. Pick segment
2. Reverse

Example:

```
[3,5,0,6] → [6,0,5,3]
```

✓ Why mutation types?

Different exploration strategies

🎯 FINAL UNDERSTANDING (VERY IMPORTANT)

□ GA FLOW (WRITE THIS IN EXAM)

```
Initialize population
→ Evaluate fitness
→ Select parents
→ Apply crossover
→ Apply mutation
→ Repeat until optimal solution
```

🚀 LAST 5-MIN REVISION

🔥 MOST IMPORTANT TO REMEMBER

- Chromosome = permutation
- Fitness = non-conflicts
- Selection = choose best
- OX crossover = safest
- Swap mutation = easiest
- Loop until best